

ArduSwarm: A Self-Contained Micro-Aerial Vehicle Testbed for Experimental Swarm Flight Control

Kwaku Richter¹, Nathaniel Mailhot^{1†}, Mohammad Pirani^{1*}, Davide Spinello^{1*}

¹Department of Mechanical Engineering, University of Ottawa, Ottawa, ON, Canada
[†]frichter@uottawa.ca, [†]nmailhot@uottawa.ca, ^{*}mpirani@uottawa.ca, ^{*}dspinell@uottawa.ca

Abstract—This paper introduces ArduSwarm, an open-source research platform that enables decentralized swarming flight research by combining peer-to-peer (P2P) communication, self-contained onboard localization, and scriptable mission execution, while also supporting on-edge AI inference using off-the-shelf, widely available Bitcraze micro-drone hardware with the ArduPilot flight control stack. Key contributions include: (i) a multi-processor architecture in which an onboard companion computer handles high-level mission scripting, communication and coordination; while the resource-constrained STM32 microcontroller conducts flight stabilization and infrastructure-free state estimation, and (ii) a dual-path communication pipeline that enables high-bandwidth telemetry to monitoring stations and low-latency peer-to-peer (P2P) mesh networking without ground station reliance. Experimental results demonstrate the platform’s self-localization capabilities in indoor flight and inter-drone coordination with the proposed P2P communication protocol. Onboard localization agrees with motion-capture ground truth, with a 0.35 m RMSE in position. Commands between drones are shown to be transmitted directly over the P2P link, achieving a median end-to-end decentralized control latency of 333 ms. The platform provides a self-contained testbed for investigating autonomous, decentralized swarming behaviors suitable for low-risk, indoor flight. Project documentation and code are available at the repository <https://github.com/kwakurichter/ArduSwarm>.

Keywords—Autonomous aerial vehicles; Swarm robotics; Multi-agent systems; Decentralized control; Cooperative systems; Wireless mesh networks; Peer-to-peer computing; Indoor navigation; Dead reckoning; Sensor fusion; Real-time systems; Embedded software.

I. INTRODUCTION

The deployment of Micro-Aerial Vehicles (MAVs) offers research potential for the study of coordinated, decentralized robotic swarming operations. Unlike single drones, swarms provide redundancy and let a single operator scale autonomous operation from one system to many systems simultaneously

[1]. However, the transition from centrally controlled coordinated flight to decentralized autonomous deployments is currently hindered by the lack of accessible research platforms that are suitable for use in a lab. Researchers are forced to choose between highly capable but large and expensive drones designed for outdoor flight, and accessible micro-drones that rely on tailored flight stacks and centralized computation to provide high-level autonomous features, thereby bypassing the complexity inherent to peer-to-peer (P2P) networks.

This paper introduces *ArduSwarm*, an open-source research platform that enables the study and implementation of decentralized and autonomous indoor swarming flight. This architecture couples the ArduPilot flight control stack with the widely available off-the-shelf Bitcraze Crazyflie micro-copter hardware ecosystem [2], [3]. This system shifts the operational paradigm from centralized control, where a ground station is required to command the drone, to a decentralized architecture where high-level coordination and mission logic is managed by individual agents via their onboard companion computers.

A. Motivation and Challenges

Indoor swarming research based on micro-copters like the Crazyflie is mostly implemented under centralized control, relying on a central hub for state estimation and agent control [4], and therefore is not suitable for the experimental challenges faced by true decentralized systems. While effective for motion planning research and coordinated flight, the centralized approach introduces a single point of failure and communication bottlenecks, limits the study of distributed coordination, inter-agent communication, and onboard self-state estimation inherent to true distributed multi-agent systems.

A key obstacle for fully distributed micro-swarms is that porting full-featured autopilots, such as ArduPilot, to micro-drones is constrained by limited onboard resources and limited support for certain subsystem driver configurations such as

This work has been partially sponsored by the National Research Council of Canada, grant 162916

radio and sensors. On platforms like the Crazyflie, most of the available CPU time, RAM, and flash budget is already consumed by real-time sensor sampling, state estimation, and attitude/rate control, leaving little headroom for higher-level mission logic, agentic reasoning, and P2P interaction.

B. Related Works

The development of aerial swarm testbeds generally entails a trade-off between the number of agents (scalability) and the autonomy of individual agents. This work sits at the intersection of micro swarm platforms, autonomous navigation, and resource-constrained flight control.

The BitCraze Crazyflie 2.1 has become a widely used platform for indoor drone flight research due to its small form factor and open architecture. The work in [5] demonstrates the simultaneous operation of large swarms (up to 49 agents) consisting of Crazyflies. Their solution relies on a centralized architecture where a ground station computes trajectories and an external motion capture system (e.g. OptiTrack or Vicon [6]) provides state estimation to agents within the swarm. The system's reliance on external infrastructure for state estimation limits its applicability towards studies where the swarm must be explicitly decentralized in its operation. The recent extension in [7] leverages ROS2 [8] to improve the modularity of the solution. Like its predecessor, real-world experiments depend on centralized management and external positioning systems for localization thus cannot enable fully decentralized swarming autonomy.

The STAR framework utilizes the Crazyflie equipped with the BitCraze AI Deck companion computer [9]. This solution replaces active motion capture with passive fiducial markers placed at known locations in the environment. While this reduces infrastructure cost relative to motion capture systems, it still requires external setup and calibration of the flight environment, unlike the infrastructure-free approach proposed in our work.

Decentralized swarms that operate without external infrastructure have been demonstrated on larger platforms. The work *EGO-Swarm* presents a fully autonomous system capable of navigation in cluttered environments using onboard computation for trajectory planning and collision avoidance [10]. Similarly, [11] demonstrate decentralized swarms for search and rescue using visual detection without explicit communication with a centralized station.

These systems rely on larger multi-rotors capable of lifting heavy sensor suites (such as LiDAR or stereo cameras) and powerful onboard computers. Given their scale, these platforms carry too high of a risk to allow for experimental work with indoor flight, highlighting the need for a solution that offers similar decentralized capabilities at the micro-copter scale.

While custom built flight stacks are common in micro-robotics, porting industry standard autopilots such as ArduPilot to resource constrained microcontrollers remains a software optimization challenge. An initial proof-of-concept for porting ArduPilot to the Crazyflie was presented in [12]. The developer

substituted the propulsion electronics for a more powerful set than what is present with the base Crazyflie. While the results presented manual flight control, it is noted that there remains flight stability issues and a lack of autonomous features. Furthermore, the system lacked integration with companion computers or drivers for sensors such as the BitCraze optical flow deck. The system was relegated to use with an external position measurement system, preventing its use as a self-contained autonomous drone.

ArduSwarm, that is the framework proposed here, builds upon these efforts, providing a fully featured and self contained autonomous flight control implementation that includes an integrated companion computer to enable higher level autonomy features.

C. Contribution

ArduSwarm proposes a decentralized MAV swarming platform, with the following key contributions:

- **Infrastructure-free Localization:** Multi-processor architecture estimates 3D position by fusing horizontal velocity estimates based on optical flow sensor data, and altitude estimates based on Time-of-Flight (ToF) sensor data on the main STM32 flight controller. High-level coordination tasks are offloaded to a companion computer.
- **Decentralized P2P Network:** Dual-path communication pipeline comprised of a high-bandwidth telemetry link to monitoring stations and a low-latency P2P network allowing agents to share self-state estimates directly with neighbours.

II. SYSTEM ARCHITECTURE

The ArduSwarm platform is built upon a hardware and software architecture designed to balance the conflicting requirements of micro-scale flight with high-level autonomous functionality.

A. Hardware Platform and Integration

The Bitcraze Crazyflie 2.1 Brushless micro-quadrotor was selected as the base hardware platform and can be augmented with expansion decks. The full setup shown in Figure 1 includes:

- **Flow Deck v2:** Provides horizontal velocity estimation via a PMW3901 optical flow sensor and altitude measurement via a VL53L1x ToF sensor.
- **AI Deck:** Features a GAP8 RISC-V multi-core processor for high-level logic and AI model inference, and an ESP32 for WiFi telemetry.
- **Micro SD Card Deck:** Enables high frequency logging of state and sensor data, which is critical for post-flight analysis and tuning.

We designed a 3D-printed protective guard that improves upon the durability of the stock propeller guards that grants better protection for the companion computer. The design integrates rigid mounting points for six passive retro-reflective markers for sub-millimeter tracking via an OptiTrack motion capture system used for ground truth benchmarking. Although

the guard adds approximately 15 g to the airframe, the total takeoff weight remains approximately 60 g, such that the system maintains a 2:1 thrust-to-weight ratio to ensure sufficient control authority.

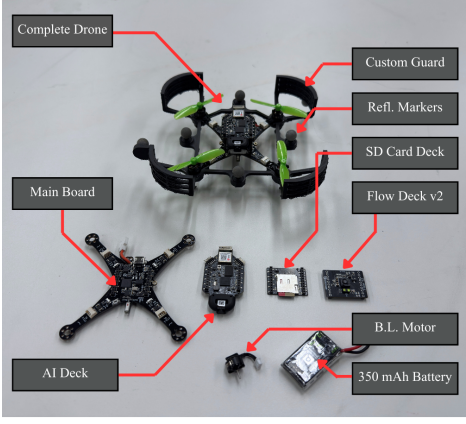


Figure. 1. Base ArduSwarm unit. Consists of unmodified, widely available BitCraze hardware (shown in lower half of photo).

B. Multi-Processor Architecture

A central challenge in porting industry standard flight software to micro-drones is the limited resource constraint of the flight controller hardware. The Crazyflie’s primary MCU, an STM32F405, possesses only 1 MB of Flash memory and 192 kB of RAM. This is insufficient to host the full ArduPilot feature set alongside complex swarm coordination algorithms.

To solve this, ArduSwarm implements a multi-processor architecture that decouples low-level flight control from high-level mission execution logic, as shown in Figure 2. The main components are:

- **Flight Controller (STM32):** Runs an optimized build of ArduCopter 4.7. It is responsible for real-time tasks including state estimation by sensor fusion, attitude stabilization, and dynamic notch filtering. Non-essential features (e.g., certain driver backends and flight modes) were disabled to fit the firmware within the 1 MB limit.
- **Companion Computer (AI Deck):** The GAP8 processor executes high-level swarm logic, processes incoming P2P messages, and computes target setpoints.
- **Data Bridge:** A UART bridge connects the two processors. The AI Deck communicates with the flight controller using the MAVLink protocol, effectively functioning as an onboard MAVLink Router that manages data flow between the flight core, the WiFi telemetry link, and the swarm mesh.

C. Communication Pipelines

To address the conflicting requirements of high-bandwidth monitoring and low-latency decentralized coordination, we developed a dual-path communication architecture, illustrated in Figure 3. This approach prevents saturation of the limited inter-agent radio link while maintaining a rich telemetry stream for the researcher.

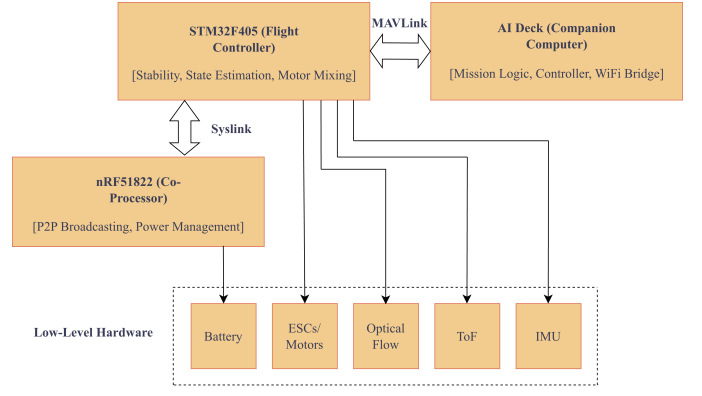


Figure. 2. Compute architecture of the ArduSwarm platform. Each drone houses three main processors which each handle different layers of the flight stack (i.e. nRF51→low-level, STM32→middle-level, AI Deck→high-level).

1) *High-Bandwidth Telemetry (AI-Deck):* The primary link for system monitoring and mission command is routed through the AI Deck’s ESP32 module. Unlike the legacy Crazyflie architecture which routes telemetry through the nRF51 radio, ArduSwarm establishes a telemetry bridge through the AI Deck. The AI Deck operates as a transparent bridge, encapsulating the full MAVLink serial stream into UDP packets over WiFi. A Python interface on the ground station acts as a proxy, receiving the UDP stream and forwarding it to standard MAVLink-compatible ground control stations. This link allows for the upload of commands, mission waypoints, and real-time parameter tuning without consuming bandwidth on the swarm’s mesh network.

2) *Decentralized Peer Network (nRF51):* The core swarm coordination is handled by the MAVLink-over-ESB pipeline running on the onboard nRF51 radio co-processor. This pipeline was developed to overcome the 32 B payload limit of the nRF51’s Enhanced ShockBurst (ESB) protocol (when used with the legacy nRF24 based radio dongle), which is insufficient for standard MAVLink packets (up to 280 B). We implemented a layered fragmentation protocol with ArduPilot. Large MAVLink messages destined for peers are intercepted by the driver, split into 24 B chunks, and wrapped in an augmented Syslink frame. A corresponding state machine on the receiving STM32 reconstructs the full MAVLink packet from the incoming fragments. The nRF51 firmware was modified to treat these fragmented packets as P2P broadcasts. This allows any agent to broadcast its state (e.g. position, velocity, battery) with all neighbors within radio range, forming a connectionless mesh. The pipeline manages system IDs to ensure distinguishing between self-state and peer-state to prevent the interpretation of an agent’s own broadcast RF reflections. By dedicating this link solely to inter-agent coordination (rather than full ground telemetry), we minimize latency and packet loss, which is critical for time-sensitive collision avoidance and formation flight.

3) *Internal Power Link:* To maintain a unified architecture, the battery monitoring system leverages this same MAVLink infrastructure internally. The nRF51, which manages the power

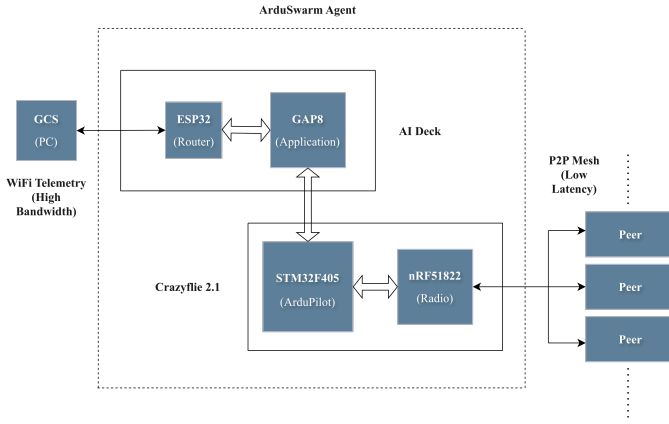


Figure 3. Dual-path communication pipeline showing the WiFi telemetry link through the AI Deck and the P2P mesh network through the nRF51 radio. Enclosed dashed lines indicate onboard hardware.

distribution, generates battery status MAVLink messages containing voltage, temperature, and charging states which are then sent to the STM32. This ensures that the flight controller reacts to voltage sags, which are common in micro-drones.

D. Hardware Abstraction Layer and Drivers

Porting ArduPilot to the Crazyflie 2.1 ecosystem required modifications to the hardware abstraction layer. The default ArduPilot drivers for components such as the optical flow sensor and rangefinder were designed for different hardware topologies or breakout boards, rendering them incompatible with the Crazyflie’s expansion decks. Furthermore, the transition to the Crazyflie 2.1 Brushless variant introduced unique electrical and resource conflicts that required new driver implementations.

1) *Optical Flow and Ranging*: To enable infrastructure-free navigation, we integrated the Bitcraze Flow Deck v2. However, the existing ArduPilot drivers for the PMW3901 optical flow sensor and VL53L1x Time-of-Flight (ToF) sensor were non-functional on this hardware. We wrote new drivers for both sensors, providing stable flow and altitude data for the onboard state estimator.

2) *Virtual Battery Monitor*: Micro-drones are highly susceptible to voltage sag, where the non-linear discharge curve of the small 1S LiPo battery can cause sudden thrust loss. Since Crazyflie delegates power management to the separate nRF51 microcontroller, the main flight controller (STM32) has no direct access to battery voltage.

The nRF51 firmware was modified in order to monitor the battery state. The power management state is forwarded internally to the STM32, where we implement a listener that intercepts and feeds it into ArduPilot’s battery monitoring system. This enables the use of voltage-compensated thrust scaling, significantly improving flight stability as the battery drains during regular flight.

III. EXPERIMENTAL VALIDATION

To illustrate the key features of the ArduSwarm platform, we designed two distinct experiments targeting the platform’s

core requirements of infrastructure-free localization and low-latency decentralized communication. Experiments were conducted in a 11 m × 6 m indoor flight arena equipped with an OptiTrack motion capture system. The OptiTrack system was utilized solely for gathering ground-truth data for post-flight analysis, otherwise the drones operate entirely independently, relying exclusively on onboard sensing and computation for navigation.

A. Experiment A: Infrastructure-Free Localization

The first experiment evaluates the accuracy of the onboard state estimator that fuses the IMU, optical flow, and ToF sensor data. Onboard localization is a prerequisite for self-contained indoor flight. By executing a pre-programmed sequence of open-loop maneuvers, we isolate the performance of the localization subsystem.

A single Crazyflie 2.1 Brushless was programmed to execute a square path using RC overrides. The mission profile consisted of an automated takeoff to a 0.8 m hover, two consecutive patterns (West–South–East–North with RC overrides for durations of $1500 \mu\text{s} \pm 200 \mu\text{s}$), followed by an automated landing. This sequence generates a square-like trajectory driven by the platform’s altitude and attitude error controllers, and holds 3D position when no RC overrides are active. Throughout the flight, the drone’s estimated position (computed onboard via Optical Flow and ToF) was logged at 20 Hz to the onboard SD card. Simultaneously, the OptiTrack system recorded the drone’s true position at 100 Hz. The performance is evaluated by calculating the Root Mean Square Error (RMSE) between the onboard state estimate and the ground truth trajectory on the NED (North-East-Down) frame. This metric quantifies the drift accumulation inherent in dead-reckoning systems and validates the stability of the optical flow driver tuning.

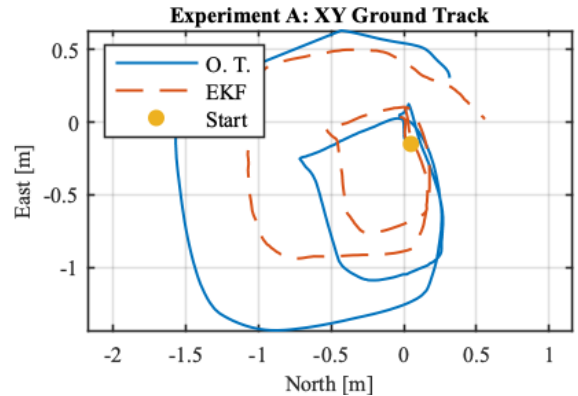


Figure 4. Onboard EKF horizontal position state estimate (XY/NE plane) vs OptiTrack reference after 0.12 s time alignment and 3 s tare. Residual indicates state estimation drift.

Since the onboard state estimate and OptiTrack streams are recorded on independent timing paths, a constant time offset may exist between the logged trajectories. We estimate this offset by sweeping a constant shift and maximizing the correlation between onboard horizontal velocity state estimate

and the OptiTrack-derived velocity. We then interpolate the time-aligned OptiTrack positions onto the onboard estimate timestamps for error computation. The estimated offset was 0.12 s. The time-aligned horizontal trajectories are compared in Figure 4, illustrating drift accumulation in the dead-reckoned estimate. The resulting 3D localization error over time is shown in Figure 5, and the corresponding position component estimates are shown in Figure 6. After a 3 s origin tare during the initial hover, the resulting horizontal localization error was $\text{RMSE}_{3\text{D}} = 0.353$ m ($\text{RMSE}_N = 0.235$ m, $\text{RMSE}_E = 0.250$ m, $\text{RMSE}_H = 0.085$ m) with median 0.330 m and 95th percentile 0.577 m.

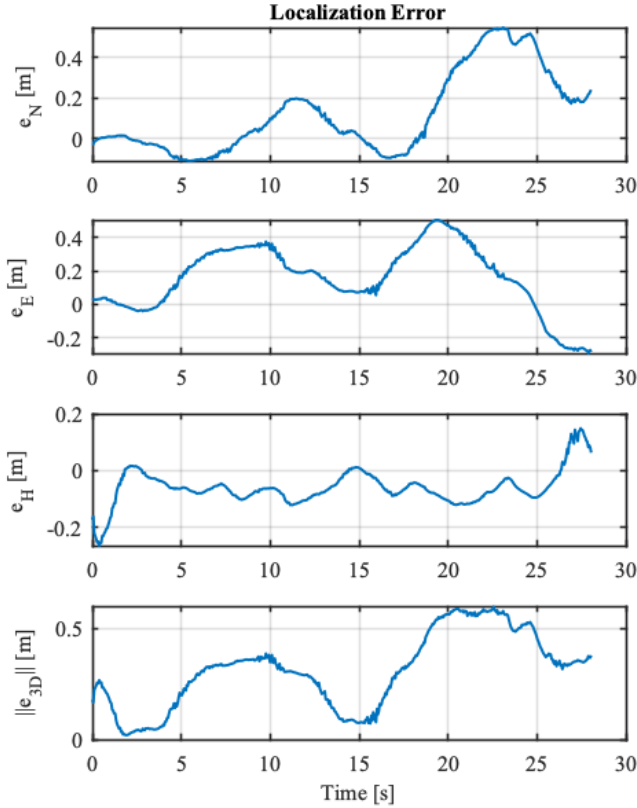


Figure 5. 3D localization error vs time. EKF–OptiTrack errors in N/E/Height and 3D norm, with noted 3D RMSE = 0.353 m, median = 0.330 m, 95th percentile = 0.577 m.

B. Experiment B: Decentralized P2P Control (Voodoo)

The second experiment, referred to as *Voodoo*, demonstrates a scenario where the attitude from a stationary agent is interpreted as a flight command by a flying agent. This experiment validates the throughput and latency of the P2P communication pipeline.

Two agents were deployed for this experiment in a leader-follower setting. The leader is a drone held by a human operator. This agent remains disarmed but actively streams its attitude at 10 Hz via the nRF51 P2P broadcast. The follower is an autonomous agent holding its position, listening for P2P broadcasts from the leader. Upon receiving a packet, the AI

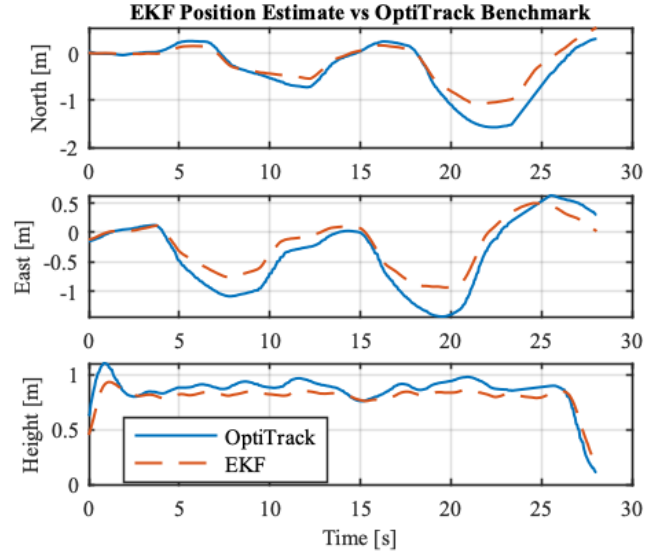


Figure 6. Components (North, East, and Height) of the onboard position state estimate and the OptiTrack reference during the open-loop maneuver. OptiTrack data are interpolated onto the onboard estimate timestamps after compensating the constant time offset (estimated at 0.12 s via horizontal velocity correlation), and both trajectories are tared using the mean position over the first 3 s of hover.

Deck decodes the leader’s attitude and maps it to a discrete, threshold-based control policy. The AI Deck acts as a virtual pilot, sending RC override commands directly to the flight controller.

The follower applies a single-action priority (pitch commands take precedence over roll commands), ensuring that only one axis is commanded at a time. Two failsafes are implemented to support decentralized operation: (i) If no P2P packets are received for 3 s, the agent initiates a landing sequence, (ii) If the optical-flow quality metric falls below 10 (texture loss) or the estimated horizontal velocity exceeds safety thresholds continuously for 0.5 s, the agent lands to prevent fly-aways. A sample of the Voodoo experiment is shown in Figure 7. The follower maps the leader’s attitude to discrete RC overrides using a $\pm 20^\circ$ deadband. Pitch angles $\theta_{\text{leader}} < -20^\circ$ trigger forward motion (1300 μs), while $\theta_{\text{leader}} > 20^\circ$ triggers backward motion (1700 μs). Similarly, roll inputs ϕ_{leader} command left (1300 μs) or right (1700 μs) movement at the same thresholds. Angles within the deadband command a stable hover (1500 μs). We define end-to-end latency as $\Delta t_{E2E} = t_{\text{vel}} - t_{\text{event}}$, where t_{event} is the leader’s first $\pm 20^\circ$ threshold crossing (command trigger) and t_{vel} is the follower’s first sustained velocity response above 0.12 m s^{-1} in the commanded direction. The measured median latency was 333 ms based on embedded leader/follower timestamps.

IV. CONCLUSION AND FUTURE WORK

This paper introduced *ArduSwarm*, a self-contained micro-MAV testbed that brings ArduPilot-class autonomy to resource-constrained hardware via a multi-processor design:

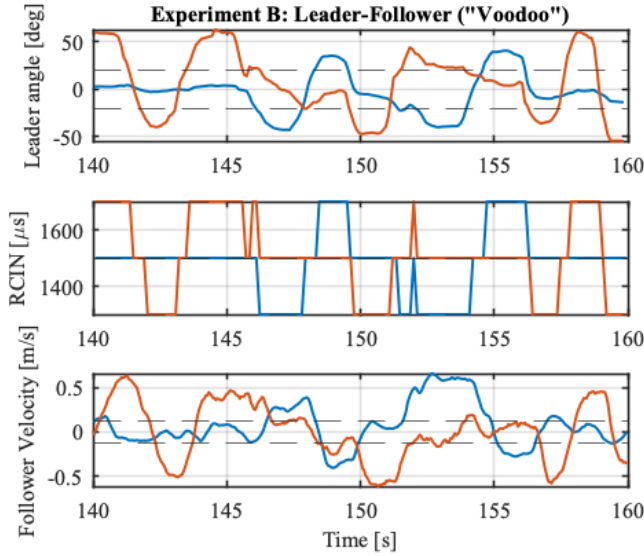


Figure 7. The leader broadcasts roll (blue) and pitch (orange) over the 10 Hz P2P link. The follower applies the threshold policy (i.e., when leader attitude exceeds the $\pm 20^\circ$ deadband) and injects a corresponding RC roll/pitch commands for durations of 1300 μ s, 1500 μ s and 1700 μ s to the flight controller. The resulting motion is shown using the follower's estimated body-frame horizontal velocity. The plotted interval excludes takeoff/landing.

ArduCopter on the flight controller for real-time state estimation and stabilization, and a companion computer for coordination and peer networking. Experiments demonstrated infrastructure-free indoor localization with 0.35 m position RMSE against OptiTrack ground truth and direct inter-agent command exchange over the proposed MAVLink-over-ESB P2P link without a central server.

Future work will use ArduSwarm to study decentralized swarming under realistic bandwidth and compute limits, where agents fuse local sensing and propagate compact state or belief information to support collective decision-making and coordinated action selection.

ACKNOWLEDGMENT

The authors thank Bitcraze AB for their donation of the Crazyflie 2.1 Brushless platform used in this study. In addition, we thank Yuvraj Cheema and Timothy Kowalski for their technical support during foundational work on incorporating ESP32 radios with the Crazyflie 2.1+ fleet, which preceded the current study.

CODE AVAILABILITY

The code and supporting materials for this work are available at: <https://github.com/kwakurichter/ArduSwarm>

REFERENCES

- [1] M. Coppola, K. N. McGuire, C. De Wagter, and G. C. H. E. de Croon, "A survey on swarming with micro air vehicles: Fundamental challenges and constraints," *Frontiers in Robotics and AI*, vol. 7, 2020. [Online]. Available: <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2020.00018>
- [2] ArduPilot Foundation, "ArduPilot," <https://ardupilot.org/>, gitHub repository, accessed 2026-01-25.

- [3] Bitcraze AB, "Bitcraze: Home," <https://www.bitcraze.io/>, accessed: 2026-01-25.
- [4] Intelligent Multi-Robot Coordination Lab, "Crazyswarm2," <https://github.com/IMRCLab/crazyswarm2>, gitHub repository, accessed 2026-01-12.
- [5] J. A. Preiss, W. Honig, G. S. Sukhatme, and N. Ayanian, "Crazyswarm: A large nano-quadcopter swarm," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 3299–3304.
- [6] Vicon Motion Systems Ltd., "Vicon Motion Capture Systems," <https://www.vicon.com/>, 2026, accessed: 2026-01-28.
- [7] L. Pichierri, A. Testa, and G. Notarstefano, "Crazychoir: Flying swarms of crazyflie quadrotors in ros 2," *IEEE Robotics and Automation Letters*, vol. 8, no. 8, pp. 4713–4720, 2023.
- [8] Open Robotics, "ROS 2 Documentation," <https://docs.ros.org/l>, 2026, accessed: 2026-01-25.
- [9] J. Chiun, Y. R. Tan, Y. Cao, J. Tan, and G. Sartoretti, "Star: Swarm technology for aerial robotics research," in *2024 24th International Conference on Control, Automation and Systems (ICCAS)*, 2024, pp. 141–146.
- [10] X. Zhou, J. Zhu, H. Zhou, C. Xu, and F. Gao, "Ego-swarm: A fully autonomous and decentralized quadrotor swarm system in cluttered environments," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 4101–4107.
- [11] J. Horyna, T. Báča, V. Walter, D. Albani, D. Hert, E. Ferrante, and M. Saska, "Decentralized swarms of unmanned aerial vehicles for search and rescue operations without explicit communication," *Autonomous Robots*, vol. 47, 10 2022.
- [12] Duccan, "Crazyflie goes ardupilot," <https://discuss.ardupilot.org/t/crazyflie-goes-ardupilot/57401/6>, 2020, blog post, accessed 2026-01-12.